ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ

ΣΧΟΛΗ ΗΜΜΥ

Τηλεπικοινωνιακά Συστήματα Ι

Παράδωση 3ης εργασίας

Ημερομηνία Παράδοσης: 23/5/2026

Ομάδα 101

Κύρκος Κωνσταντίνος 2022030112

1. For a given N (for example, N = 200), generate a binary sequence with 4N equiprobable bits.

```
%% A.1

bits = (sign(randn(4*N, 1)) + 1)/2;
```

2. (15) Write a function function X = bits to 4 PAM(bit seq,A) which, using Gray coding, maps the input binary sequence bit seq to a sequence of 4-PAM symbols.

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB3/bits_to_4PAM.m

```
function X = bits_to_4PAM(b)

    b_pairs = reshape(b, 2, [])';

    % Αρχικοποίηση του διανύσματος των συμβόλων
    X = zeros(length(b)/2, 1);

    % mapping
    idx_00 = (b_pairs(:,1) == 0) & (b_pairs(:,2) == 0);
    X(idx_00) = 3;

    idx_01 = (b_pairs(:,1) == 0) & (b_pairs(:,2) == 1);
    X(idx_01) = 1;

    idx_11 = (b_pairs(:,1) == 1) & (b_pairs(:,2) == 1);
    X(idx_11) = -1;

    idx_10 = (b_pairs(:,1) == 1) & (b_pairs(:,2) == 0);
    X(idx_10) = -3;
end
```

3. Map the first 2N bits of the sequence from Step 1 to the 4-PAM symbols $X_{I,n}$, for n = 1, . . . ,N, and the next 2N bits to the 4-PAM symbols $X_{Q,n}$, for n = 1, . . . ,N.

```
%% A.3

% For bits in the [0,2N]
bitXI = bits(1:2*N);
XIn = bits_to_4_PAM(bitXI, A);

% For bits in the [2N+1,4N]
bitXQ = bits(2*N+1:4*N);
XQn = bits_to_4_PAM(bitXQ, A);
```

4. (5) Pass the sequences $\{X_{I,n}\}$ and $\{X_{Q,n}\}$ through the SRRC pulse-shaping filters. Assume a symbol period $T = 10^{-3}$ sec, over = 10, $T_s = T_{over}$ and construct and plot the output waveforms $X_I(t)$ and $X_Q(t)$ (set the correct time axis), as well as their periodograms.

```
%% A.4

% Creating the pulse
[phi, t] = srrc_pulse(T, over, A_SRRC, a);

% Upsample the 2 sequencies
XIn_up = (1/Ts)*upsample(XIn, over);
XI_up_time = 0:Ts:N*T-Ts;

XQn_up = (1/Ts)*upsample(XQn, over);
XQ_up_time = XI_up_time;

% Convolutions and time
XIt = Ts*conv(XIn_up, phi);
XQt = Ts*conv(XQn_up, phi);
conv_t = t(1) + XQ_up_time(1):Ts:t(end) + XQ_up_time(end);

% Output waveform plots
figure;
plot(conv_t, XIt);
title("Output waveform XI(t)");
xlabel("t");
ylabel("XI(t)");
xlim([conv_t(1) conv_t(end)]);

figure;
plot(conv_t, XQt);
title("Output waveform XQ(t)");
xlabel("t");
ylabel("XQ(t)");
xlim([conv_t(1) conv_t(end)]);

% Frequency Range
f_axis = linspace(-Fs/2,(Fs/2-Fs/Nf), Nf);

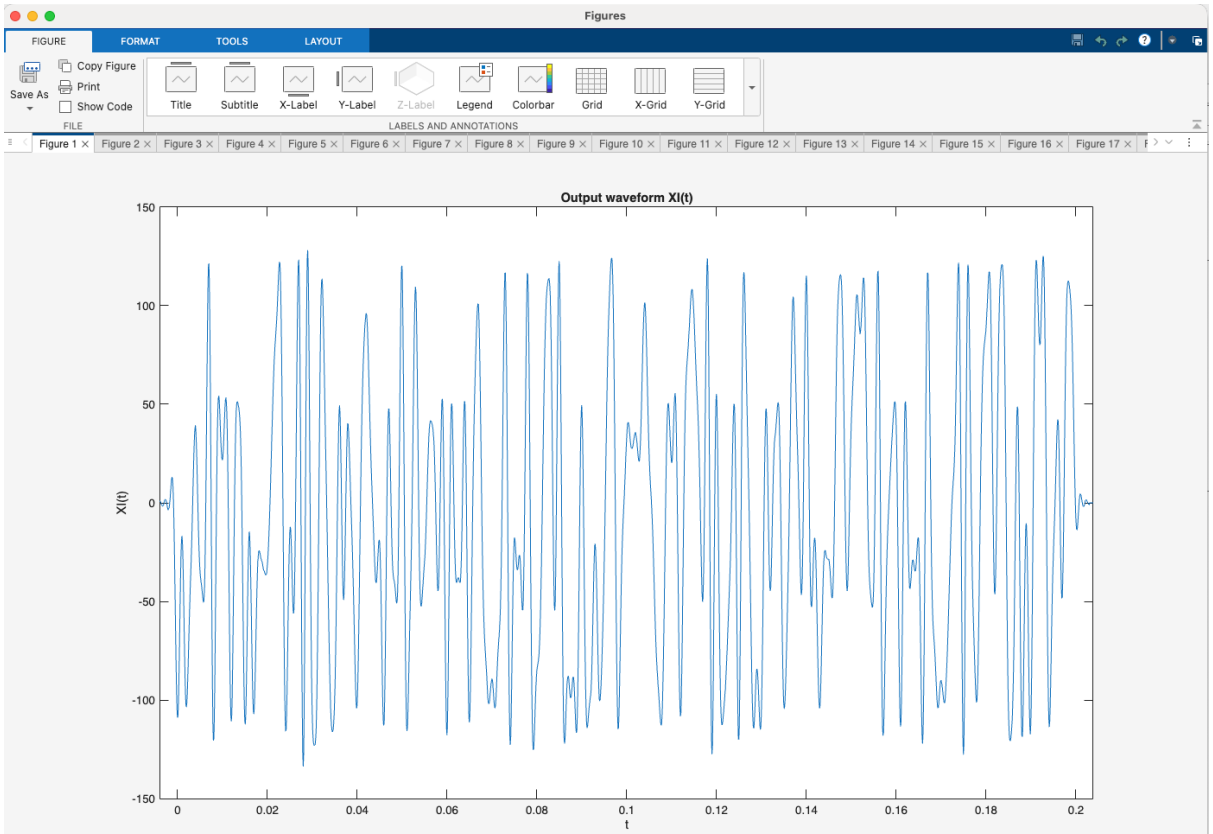
% Periodograms creation
XIF = fftshift(fft(XIt, Nf)*Ts);
num = power(abs(XIF),2);
T_total = conv_t(end) - conv_t(1);
PXIF = num/T_total;

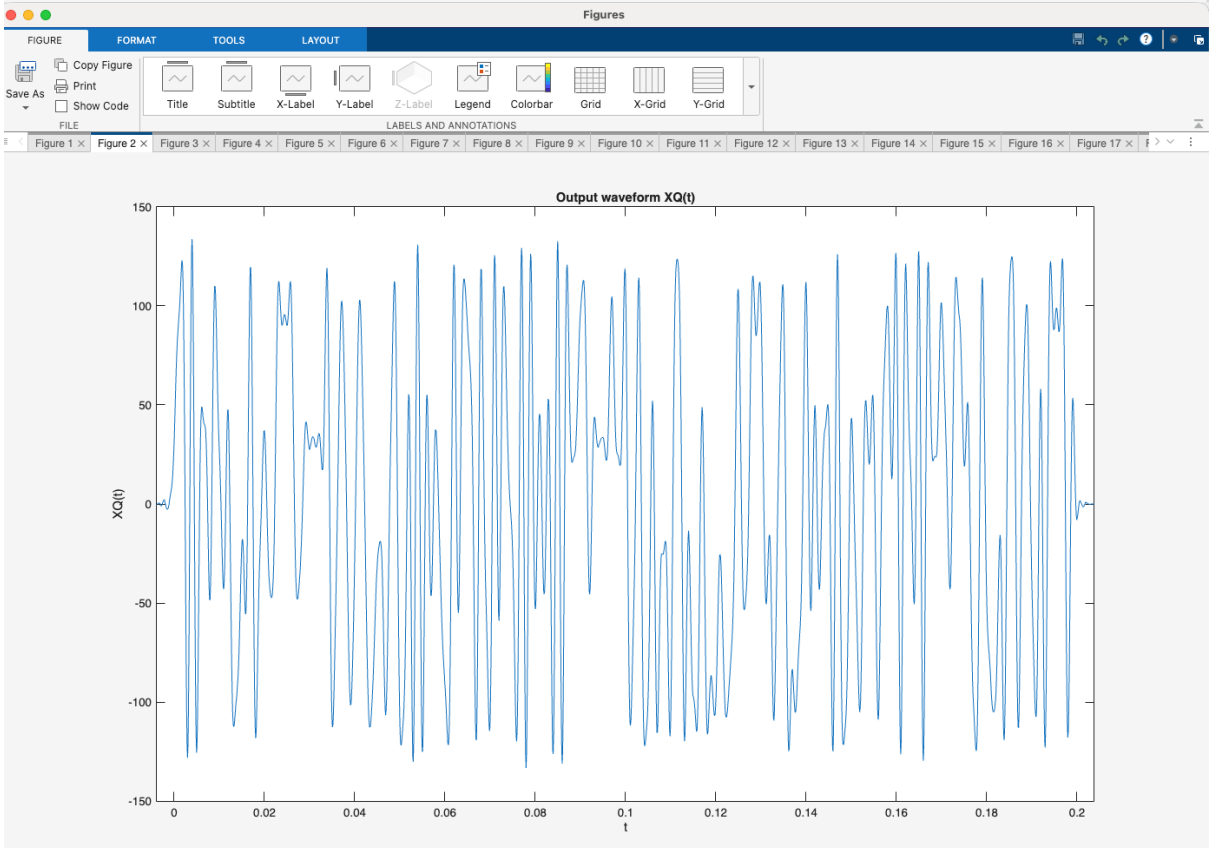
XQF = fftshift(fft(XQt, Nf)*Ts);
num = power(abs(XQF),2);
T_total = conv_t(end) - conv_t(1);
PXQF = num/T_total;

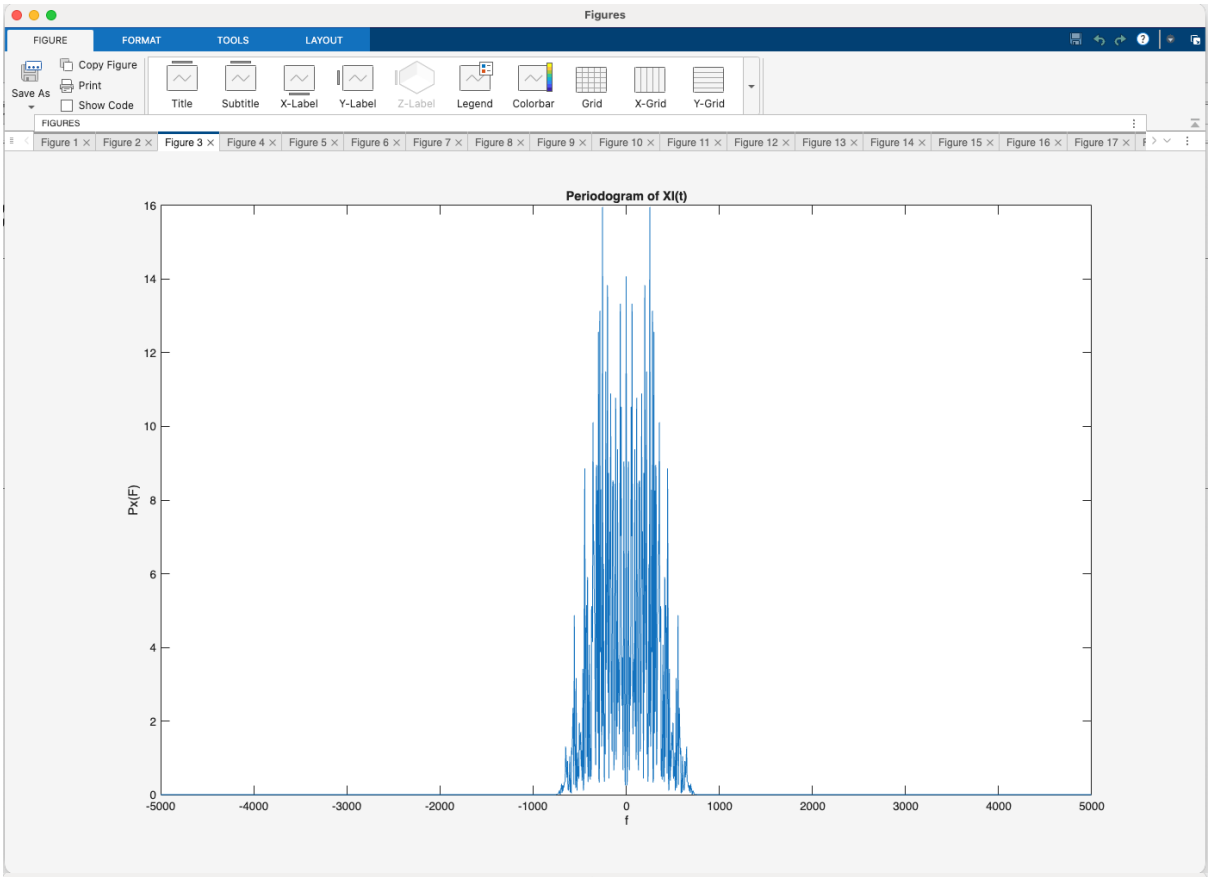
% Periodogram plots
figure;
plot(f_axis, PXIF);
title("Periodogram of XI(t)");
xlabel("f");
ylabel("Px(F)");

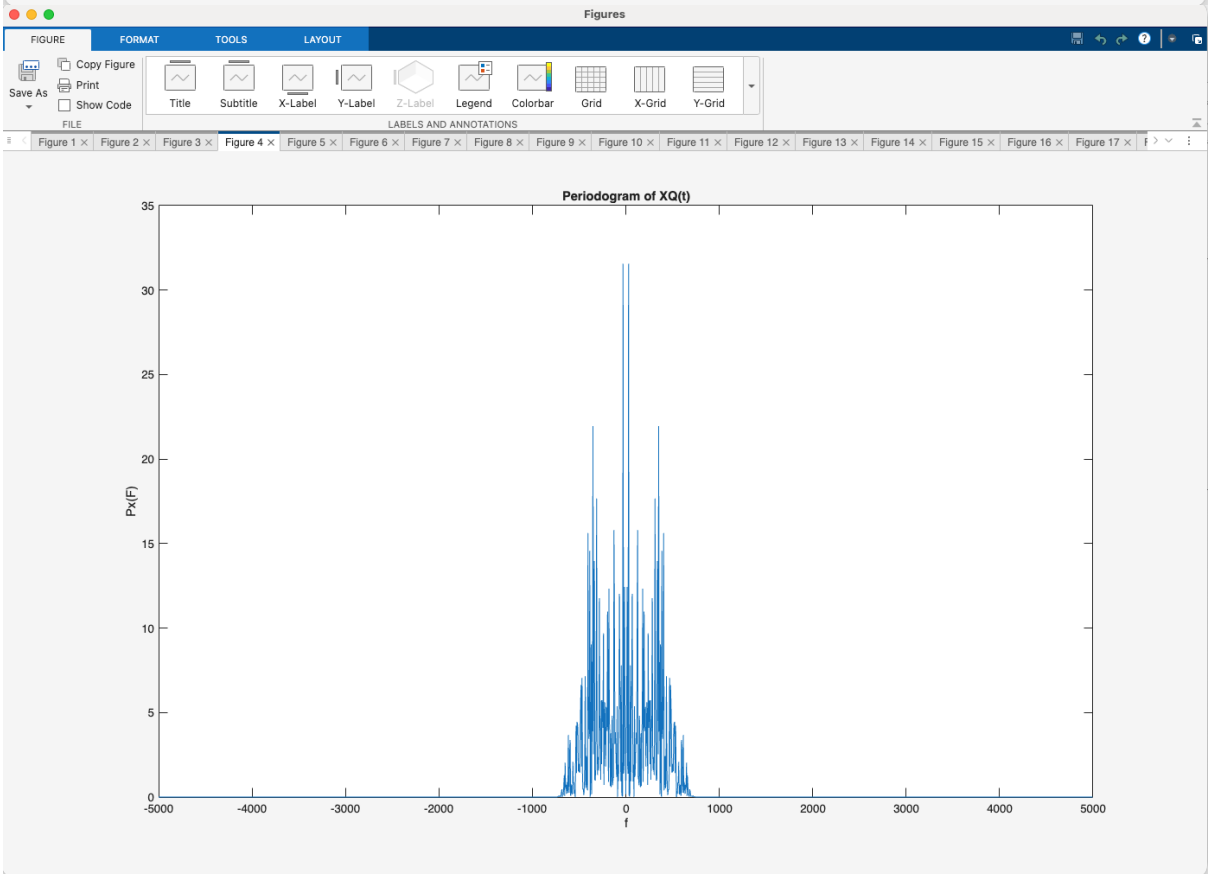
figure;
plot(f_axis, PXQF);
title("Periodogram of XQ(t)");
xlabel("f");
ylabel("Px(F)");

```


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 1
Figure 2
Figure 3
Figure 4
Figure 5
Figure 6
Figure 7
Figure 8
Figure 9
Figure 10
Figure 11
Figure 12
Figure 13
Figure 14
Figure 15
Figure 16
Figure 17
Output waveform XI(t)
XI(t)
t
150
100
50
0
-50
-100
-150
0
0.02
0.04
0.06
0.08
0.1
0.12
0.14
0.16
0.18
0.2


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 1
Figure 2
Figure 3
Figure 4
Figure 5
Figure 6
Figure 7
Figure 8
Figure 9
Figure 10
Figure 11
Figure 12
Figure 13
Figure 14
Figure 15
Figure 16
Figure 17
Output waveform XQ(t)
XQ(t)
t
150
100
50
0
-50
-100
-150
0
0.02
0.04
0.06
0.08
0.1
0.12
0.14
0.16
0.18
0.2


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
FIGURES
Figure 1
Figure 2
Figure 3
Figure 4
Figure 5
Figure 6
Figure 7
Figure 8
Figure 9
Figure 10
Figure 11
Figure 12
Figure 13
Figure 14
Figure 15
Figure 16
Figure 17
Periodogram of XI(t)
Px(F)
f
0
2
4
6
8
10
12
14
16
-5000
-4000
-3000
-2000
-1000
0
1000
2000
3000
4000
5000


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
FILE
LABELS AND ANNOTATIONS
Figure 1
Figure 2
Figure 3
Figure 4
Figure 5
Figure 6
Figure 7
Figure 8
Figure 9
Figure 10
Figure 11
Figure 12
Figure 13
Figure 14
Figure 15
Figure 16
Figure 17
Periodogram of XQ(t)
Px(F)
f
0
5
10
15
20
25
30
35
-5000
-4000
-3000
-2000
-1000
0
1000
2000
3000
4000
5000

5. (5) Multiply the waveforms $X_I(t)$ and $X_Q(t)$ by the corresponding carriers (for example, $F_0 = 2KHz$) and generate the waveforms

$X_{modI}(t) = 2X_I(t)\cos(2\ F_0t)$, $X_{modQ}(t) = -2X_Q(t)\sin(2\ F_0t)$.

Plot the resulting waveforms as well as the corresponding periodograms. What do you observe?

```
%% A.5

Fo = 2000;
XImod_t = 2*XIt.*cos(2*pi*Fo*conv_t);
XQmod_t = -2*XQt.*sin(2*pi*Fo*conv_t);

% Plots of the new waveforms
figure;
plot(conv_t, XImod_t);
title("2XI(t)cos(2πFot)");
xlabel("t");
ylabel("XI(t)");
xlim([conv_t(1) conv_t(end)]);

figure;
plot(conv_t, XQmod_t);
title("-2XQ(t)sin(2πFot)");
xlabel("t");
ylabel("XQ(t)");
xlim([conv_t(1) conv_t(end)]);

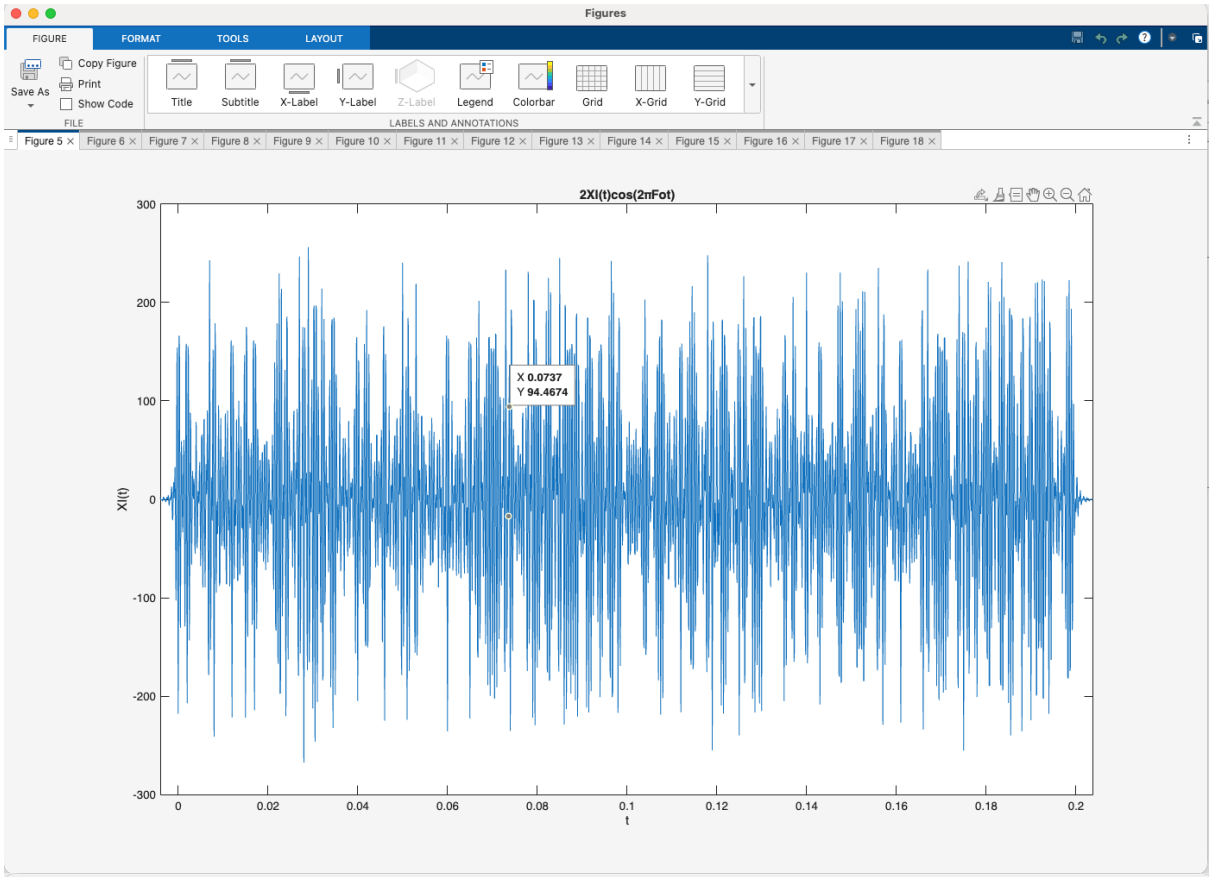
% Periodograms creation
XIFmod = fftshift(fft(XImod_t, Nf)*Ts);
num = power(abs(XIFmod),2);
T_total = conv_t(end) - conv_t(1);
PXIFmod = num/T_total;

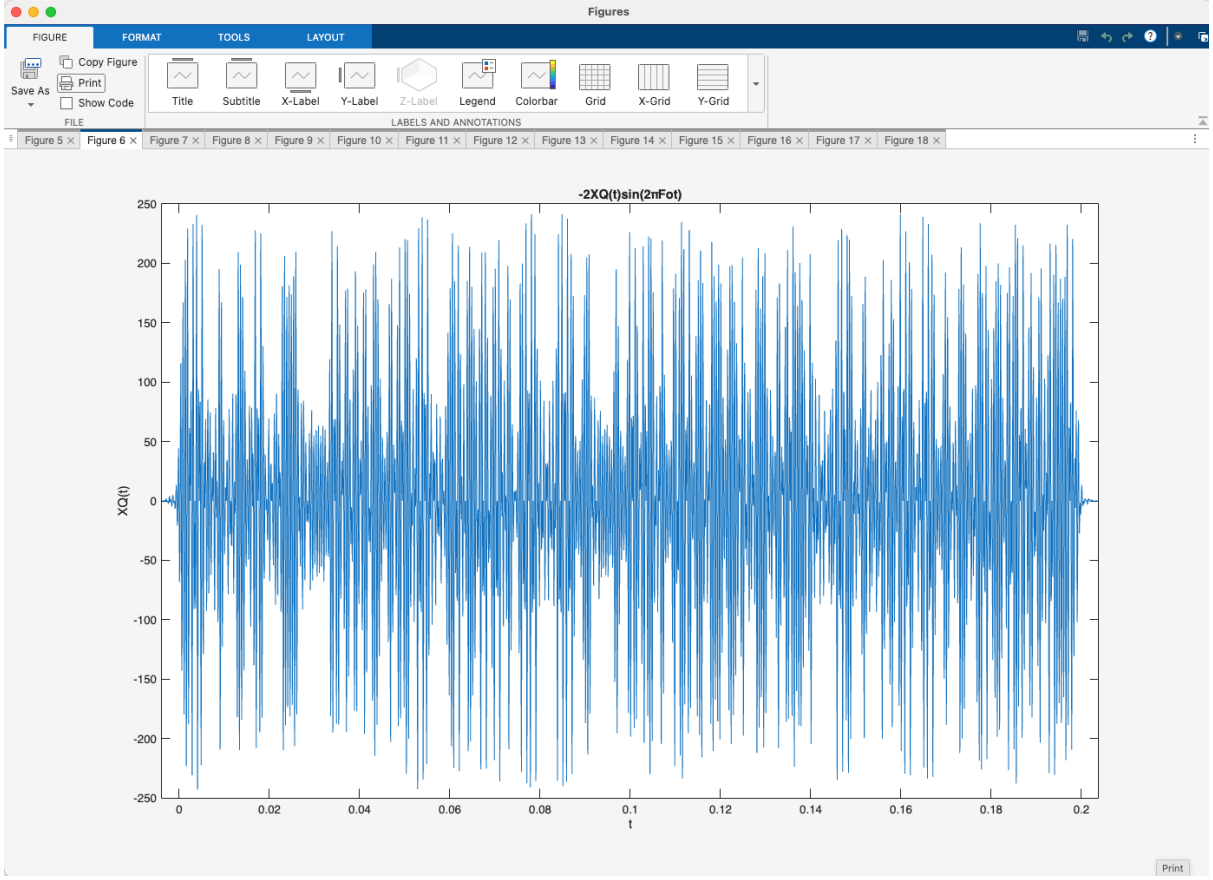
XQFmod = fftshift(fft(XQmod_t, Nf)*Ts);
num = power(abs(XQFmod),2);
T_total = conv_t(end) - conv_t(1);
PXQFmod = num/T_total;

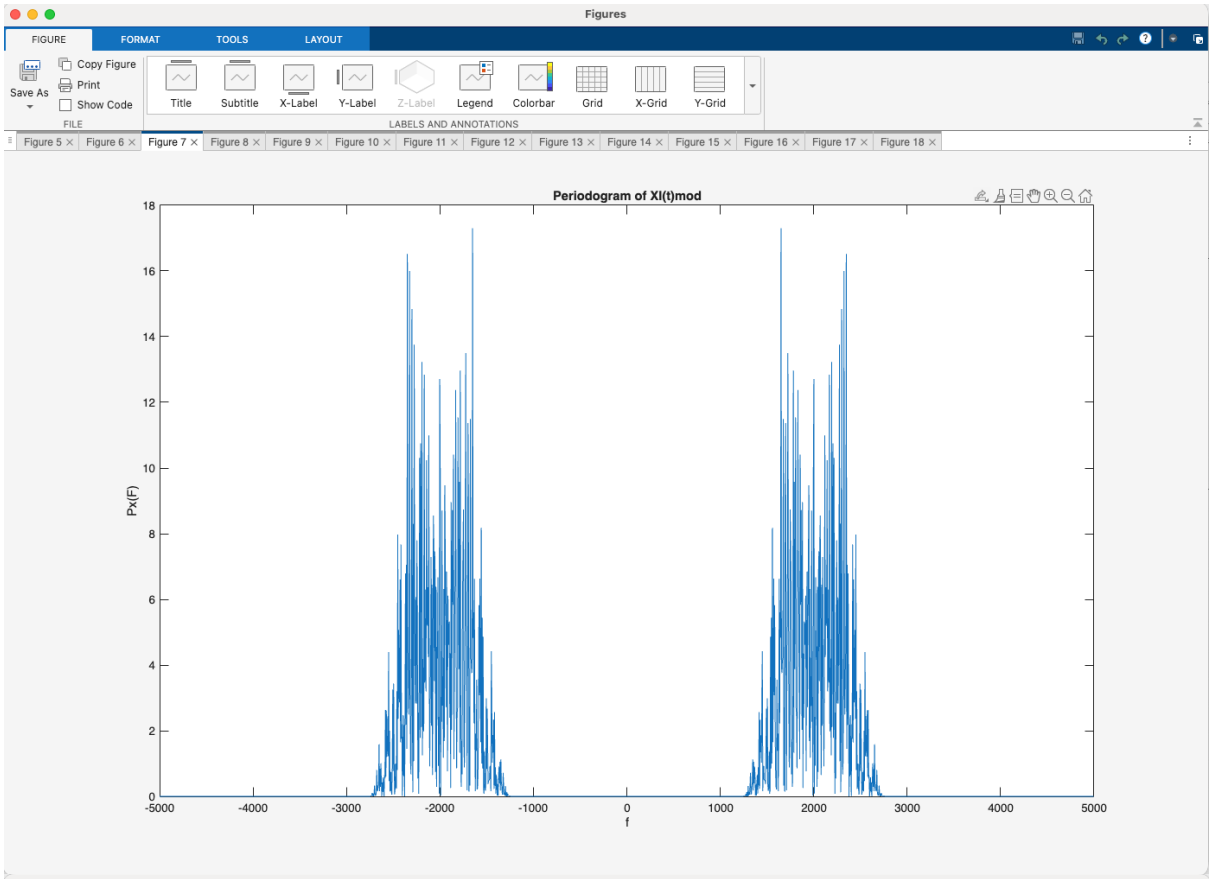
% Periodogram plots
figure;
plot(f_axis, PXIFmod);
title("Periodogram of XI(t)mod");
xlabel("f");
ylabel("Px(F)");

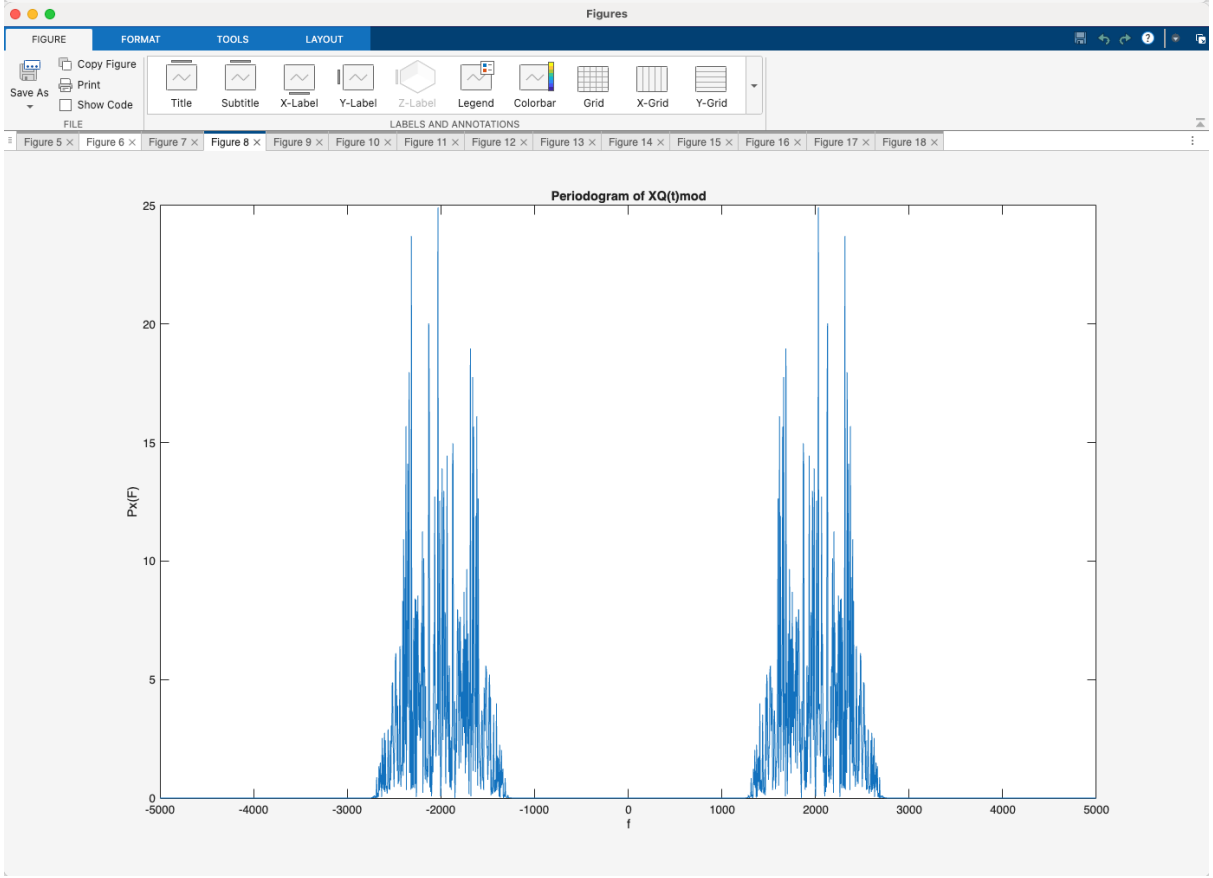
figure;
plot(f_axis, PXQFmod);
title("Periodogram of XQ(t)mod");
xlabel("f");
ylabel("Px(F)");

```


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 5
Figure 6
Figure 7
Figure 8
Figure 9
Figure 10
Figure 11
Figure 12
Figure 13
Figure 14
Figure 15
Figure 16
Figure 17
Figure 18
2XI(t)cos(2πFot)
X 0.0737
Y 94.4674
XI(t)
t


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 5
Figure 6
Figure 7
Figure 8
Figure 9
Figure 10
Figure 11
Figure 12
Figure 13
Figure 14
Figure 15
Figure 16
Figure 17
Figure 18
-2XQ(t)sin(2πFot)
XQ(t)
t
Print





As we can observe from the periodograms, the modulation has taken place around the frequency of +- 2000hz.

6. (5) Construct and plot the channel input, $X_{mod}(t) = X_{modI}(t) + X_{modQ}(t)$ and its periodogram. What do you observe?

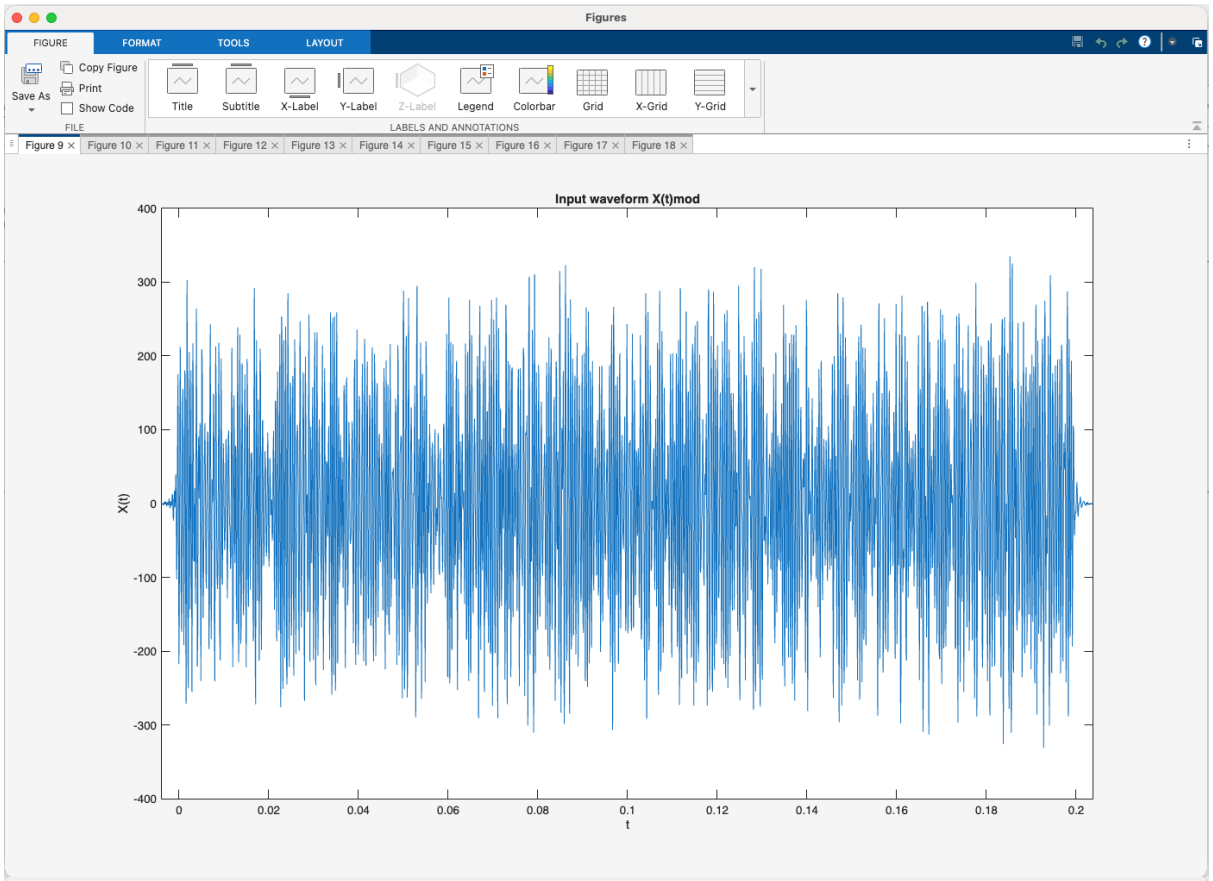
```
%% A.6

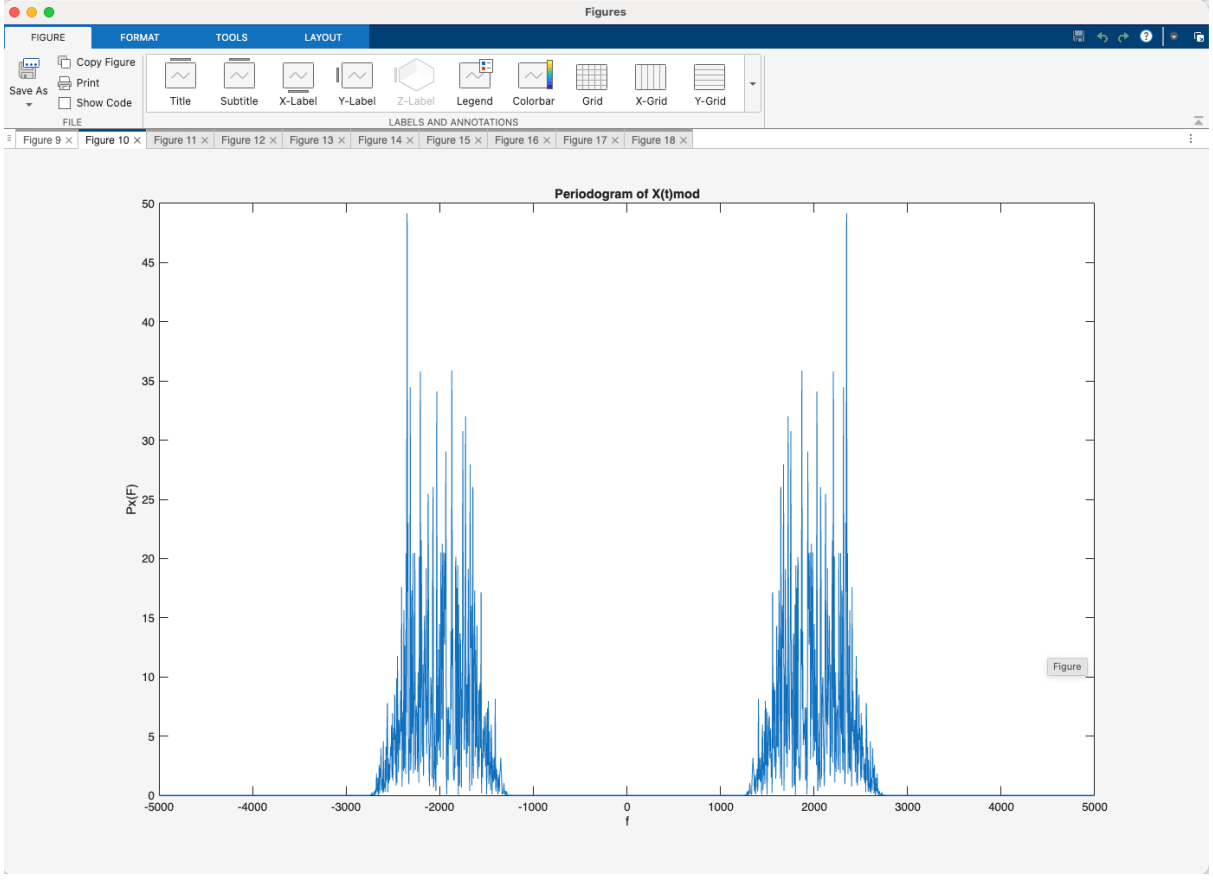
% Input signal creation
Xmod_t = XImod_t + XQmod_t;

% Plot of Xmod(t)
figure;
plot(conv_t, Xmod_t);
title("Input waveform X(t)mod");
xlabel("t");
ylabel("X(t)");
xlim([conv_t(1) conv_t(end)]);

% Periodogram creation
Xmod_F = fftshift(fft(Xmod_t, Nf)*Ts);
num = power(abs(Xmod_F),2);
T_total = conv_t(end) - conv_t(1);
PXmod = num/T_total;

% Periodogram plot
figure;
plot(f_axis, PXmod);
title("Periodogram of X(t)mod");
xlabel("f");
ylabel("Px(F)");
```





As we can observe from the periodograms, the modulation has taken place around the frequency of +- 2000hz. Also due to the addition, the amplitude has also increased.

8. (5) At the channel output, add white Gaussian noise W(t) with variance equal to

$$\sigma_W^2 = \frac{10A^2}{T_s \cdot 10^{\frac{\mathrm{SNR}_{\mathrm{dB}}}{10}}}.$$

```
159    %% A.8
160
161    %Noise creation
162    SNR = 10;
163    %SNR = 20;
164    variance = (10*(A^2))/(Ts*(10^(SNR/10)));
165    gaussian_noise = sqrt(variance)*randn(1,length(Xmod_t));
166
167    % Adding noise to the waveform
168    Xmod_noise = Xmod_t + gaussian_noise;
169
```

9. (5) At the receiver, split the noisy waveform into two branches and multiply them by the carriers cos(2 F_0t) and −sin(2 F_0t), respectively. Plot the waveforms that result and their periodograms. What do you observe?

```
%% A.9

% Demodulating the signal
XImod_noise = Xmod_noise.*cos(2*pi*Fo*conv_t);
XQmod_noise = -Xmod_noise.*sin(2*pi*Fo*conv_t);

% Plot the signals
figure;
plot(conv_t, XImod_noise);
title("XI(t)cos(2πFot)")
xlabel("t");
ylabel("XI(t)");
xlim([conv_t(1) conv_t(end)]);

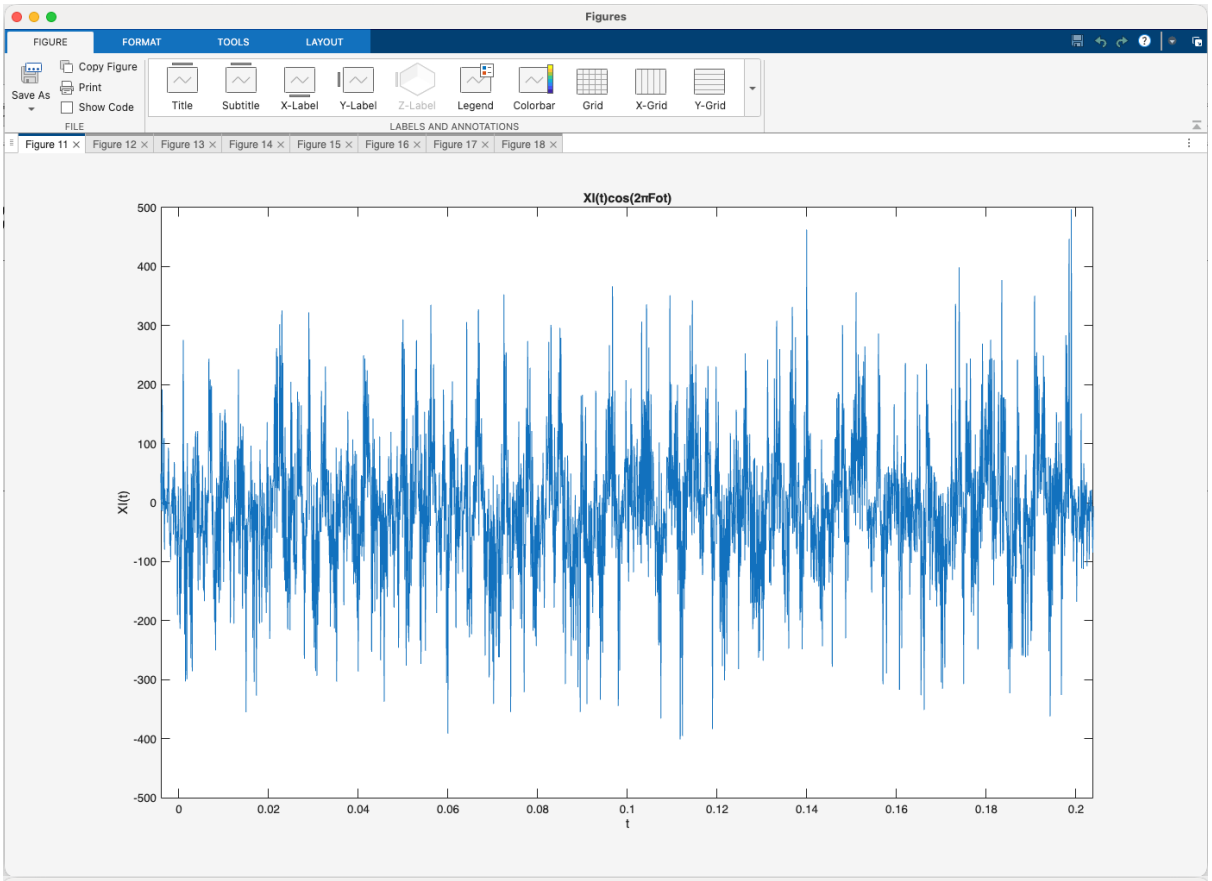
figure;
plot(conv_t, XQmod_noise);
title("-XQ(t)sin(2πFot)")
xlabel("t");
ylabel("XQ(t)");
xlim([conv_t(1) conv_t(end)]);

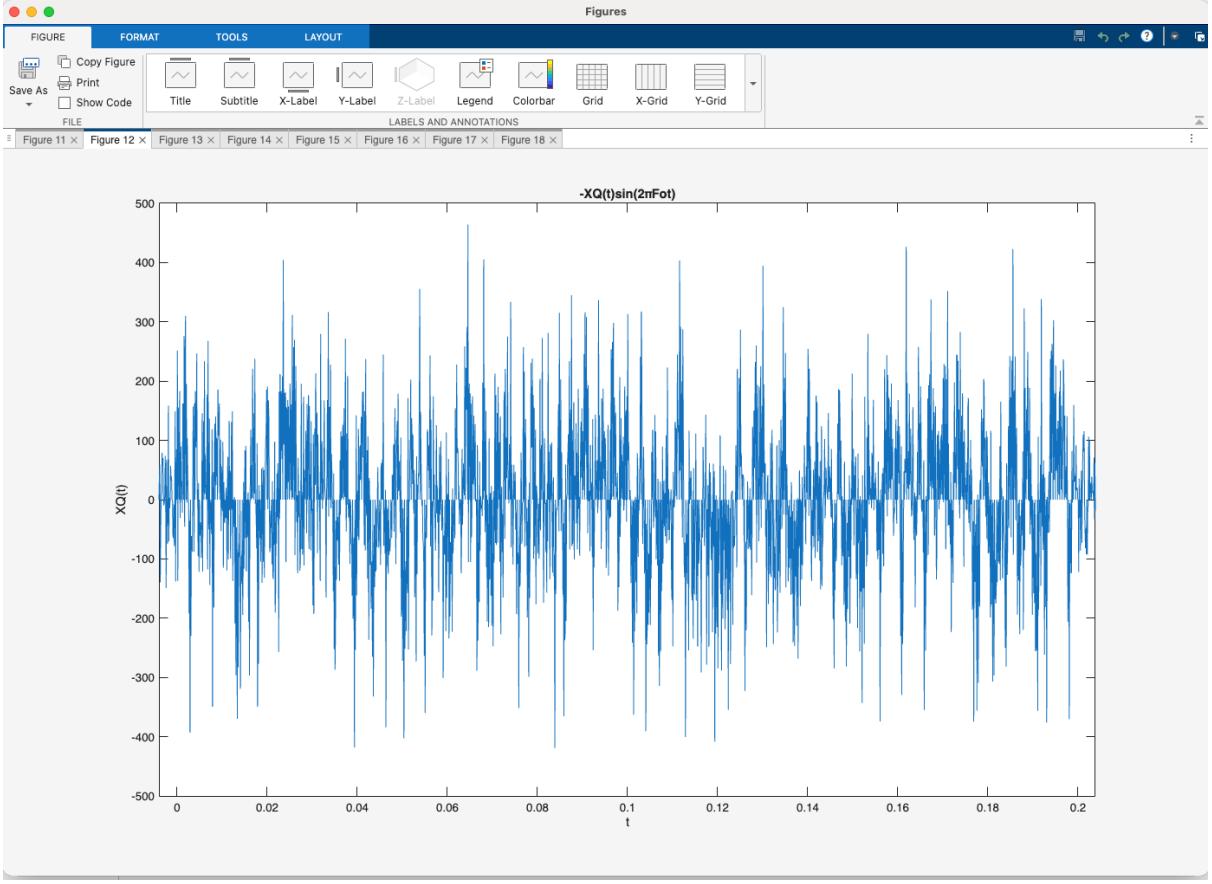
% Periodograms creation
XImod_noise_F = fftshift(fft(XImod_noise, Nf)*Ts);
num = power(abs(XImod_noise_F),2);
T_total = conv_t(end) - conv_t(1);
PXImod_noise = num/T_total;

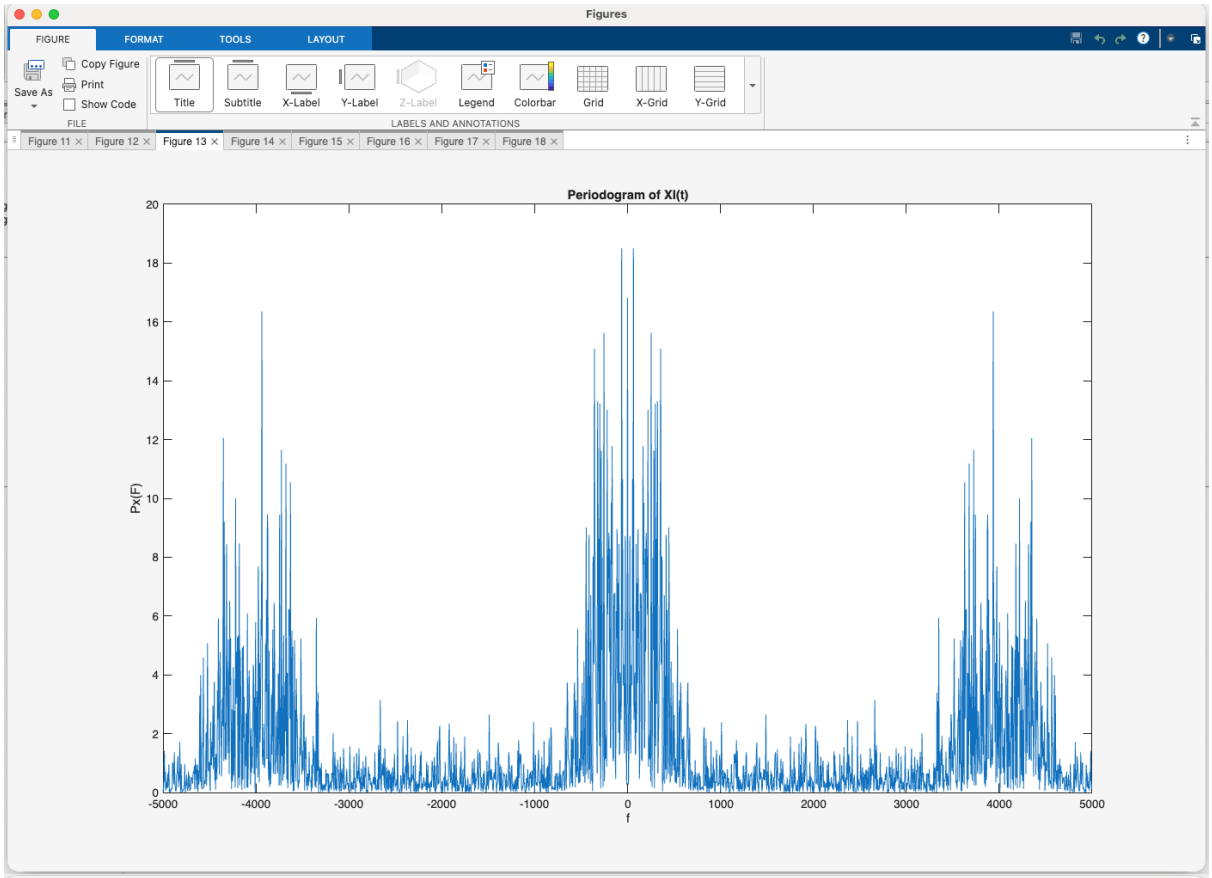
XQmod_noise_F = fftshift(fft(XQmod_noise, Nf)*Ts);
num = power(abs(XQmod_noise_F),2);
T_total = conv_t(end) - conv_t(1);
PXQmod_noise = num/T_total;

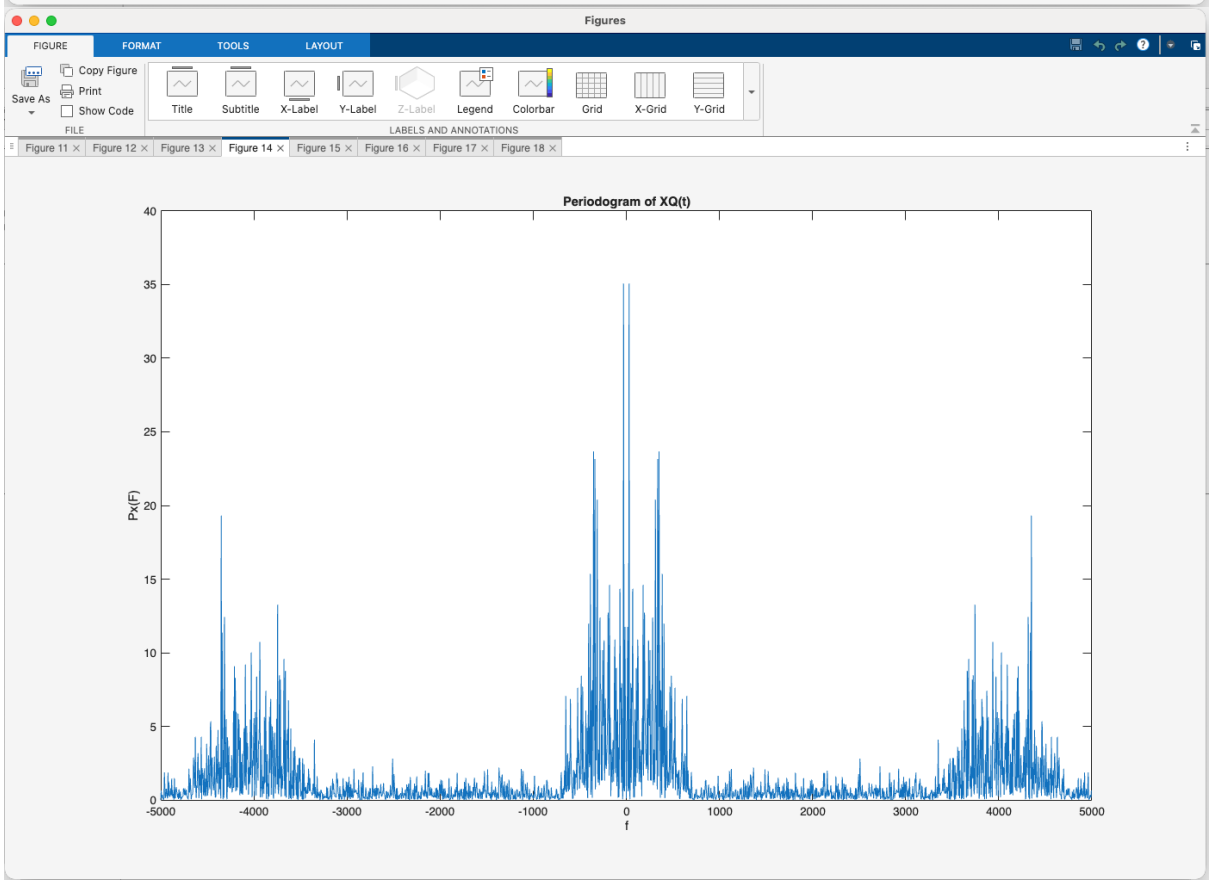
% Plot the periodogram
figure;
plot(f_axis, PXImod_noise);
title("Periodogram of XI(t)")
xlabel("f");
ylabel("Px(F)");

figure;
plot(f_axis, PXQmod_noise);
title("Periodogram of XQ(t)")
xlabel("f");
ylabel("Px(F)");
```


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 11
Figure 12
Figure 13
Figure 14
Figure 15
Figure 16
Figure 17
Figure 18
XI(t)cos(2πFot)
XI(t)
t


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 11
Figure 12
Figure 13
Figure 14
Figure 15
Figure 16
Figure 17
Figure 18
-XQ(t)sin(2πFot)
XQ(t)
t





We observe that the periodograms are centered around -F0,0,+F0

10. (5) Pass the waveforms computed in the previous step through the matched filters. Plot the resulting waveforms and their periodograms (set the correct time axis). What do you observe?

```
%% A.10

% Filter the waveforms
XIt_demodulated = Ts*conv(XImod_noise, phi);
XQt_demodulated = Ts*conv(XQmod_noise, phi);
conv_t2 = t(1) + conv_t(1):Ts:t(end) + conv_t(end);

% Plot the filtered waveforms
figure;
plot(conv_t2, XIt_demodulated);
title("Filtered XI(t)")
xlabel("t");
ylabel("XI(t)");
xlim([conv_t2(1) conv_t2(end)]);

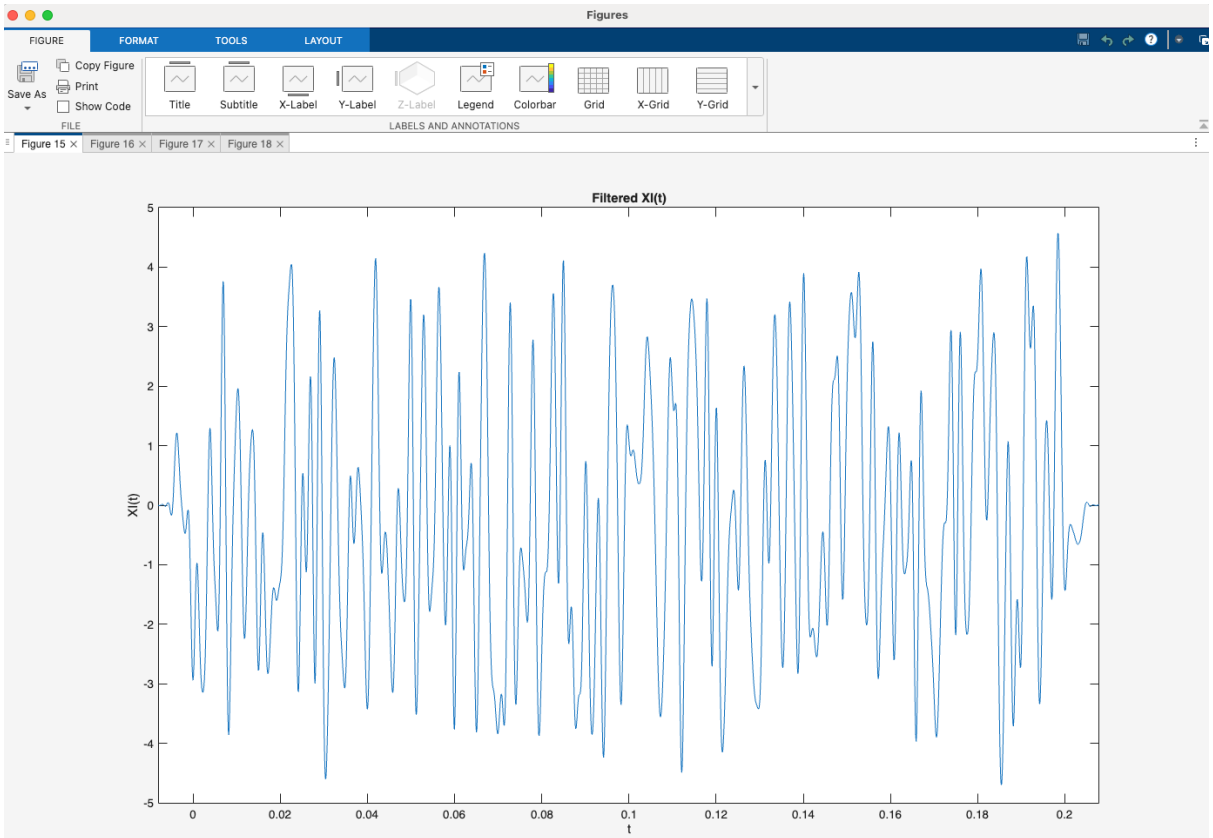
figure;
plot(conv_t2, XQt_demodulated);
title("Filtered XQ(t)")
xlabel("t");
ylabel("XQ(t)");
xlim([conv_t2(1) conv_t2(end)]);

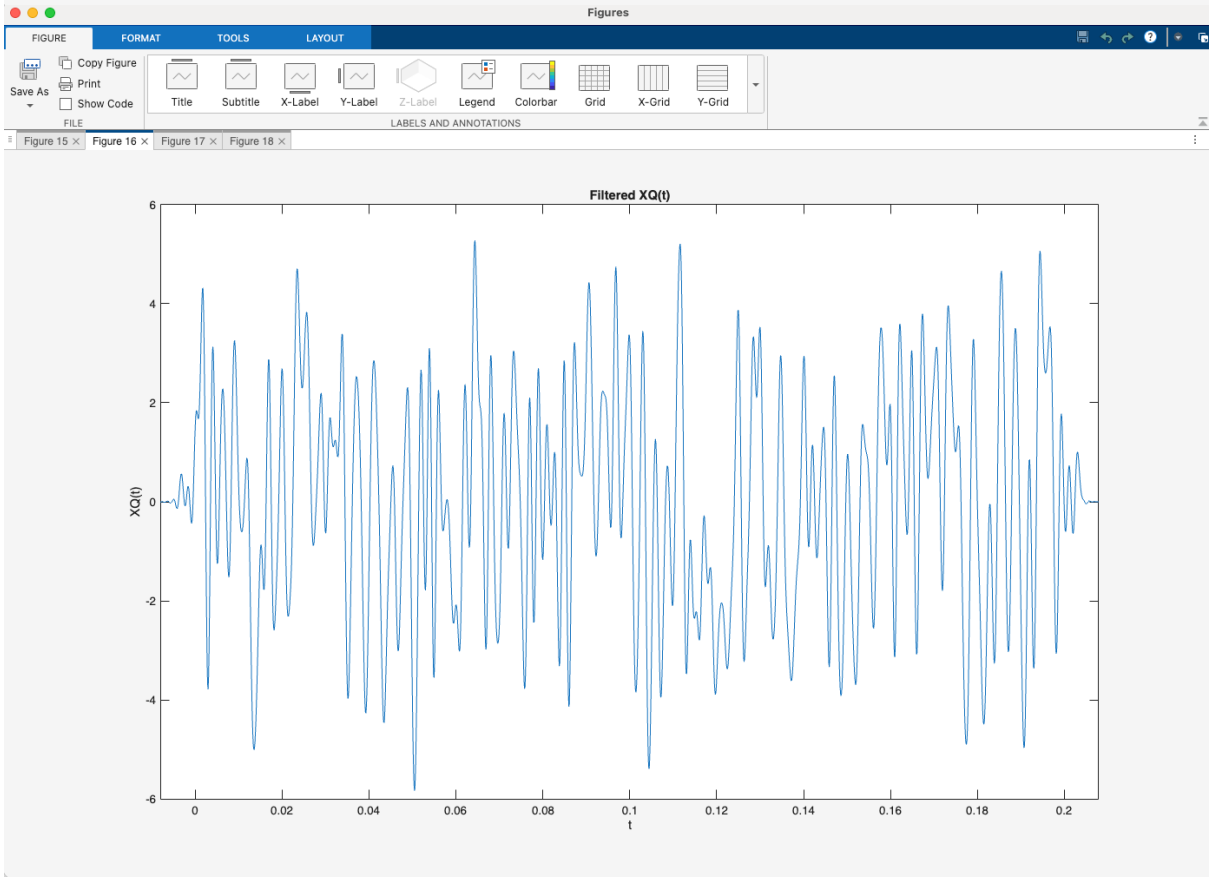
XIF_demodulated = fftshift(fft(XIt_demodulated, Nf)*Ts);
num = power(abs(XIF_demodulated),2);
T_total = conv_t2(end) - conv_t2(1);
PXI_demodulated = num/T_total;

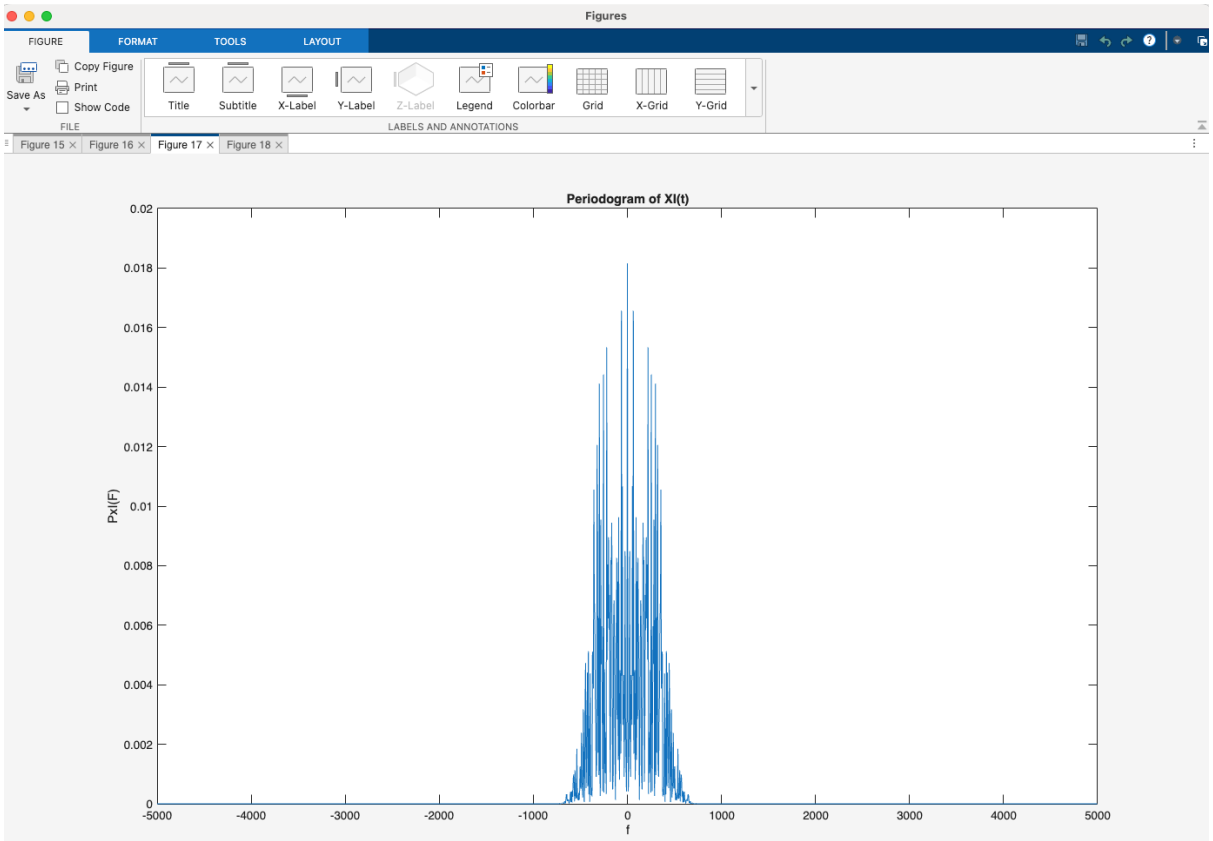
XQF_demodulated = fftshift(fft(XQt_demodulated, Nf)*Ts);
num = power(abs(XQF_demodulated),2);
T_total = conv_t2(end) - conv_t2(1);
PXQ_demodulated = num/T_total;

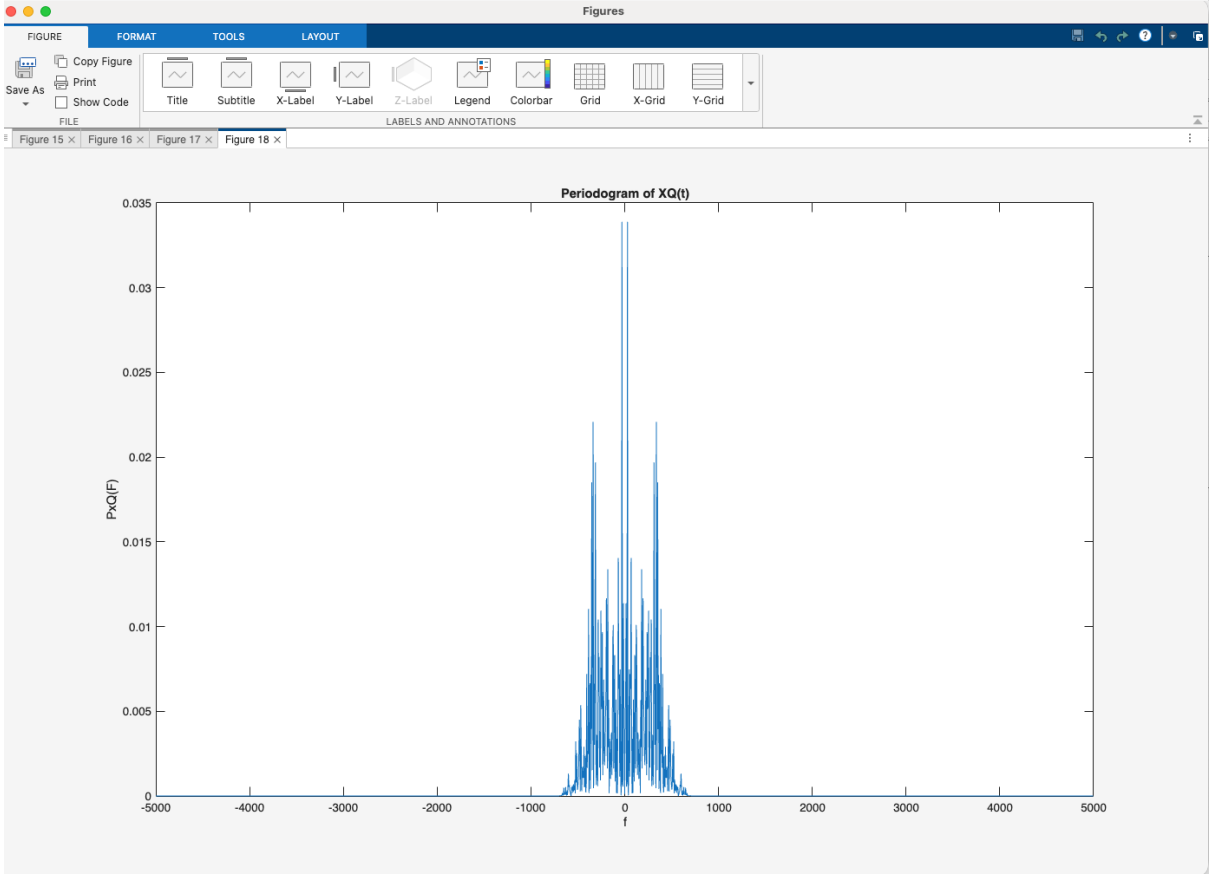
% Plot the periodogram of filtered waveforms
figure;
plot(f_axis, PXI_demodulated);
title("Periodogram of XI(t)")
xlabel("f");
ylabel("PxI(F)");

figure;
plot(f_axis, PXQ_demodulated);
title("Periodogram of XQ(t)")
xlabel("f");
ylabel("PxQ(F)");
```


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 15
Figure 16
Figure 17
Figure 18
Filtered XI(t)
XI(t)
t


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 15
Figure 16
Figure 17
Figure 18
Filtered XQ(t)
XQ(t)
t

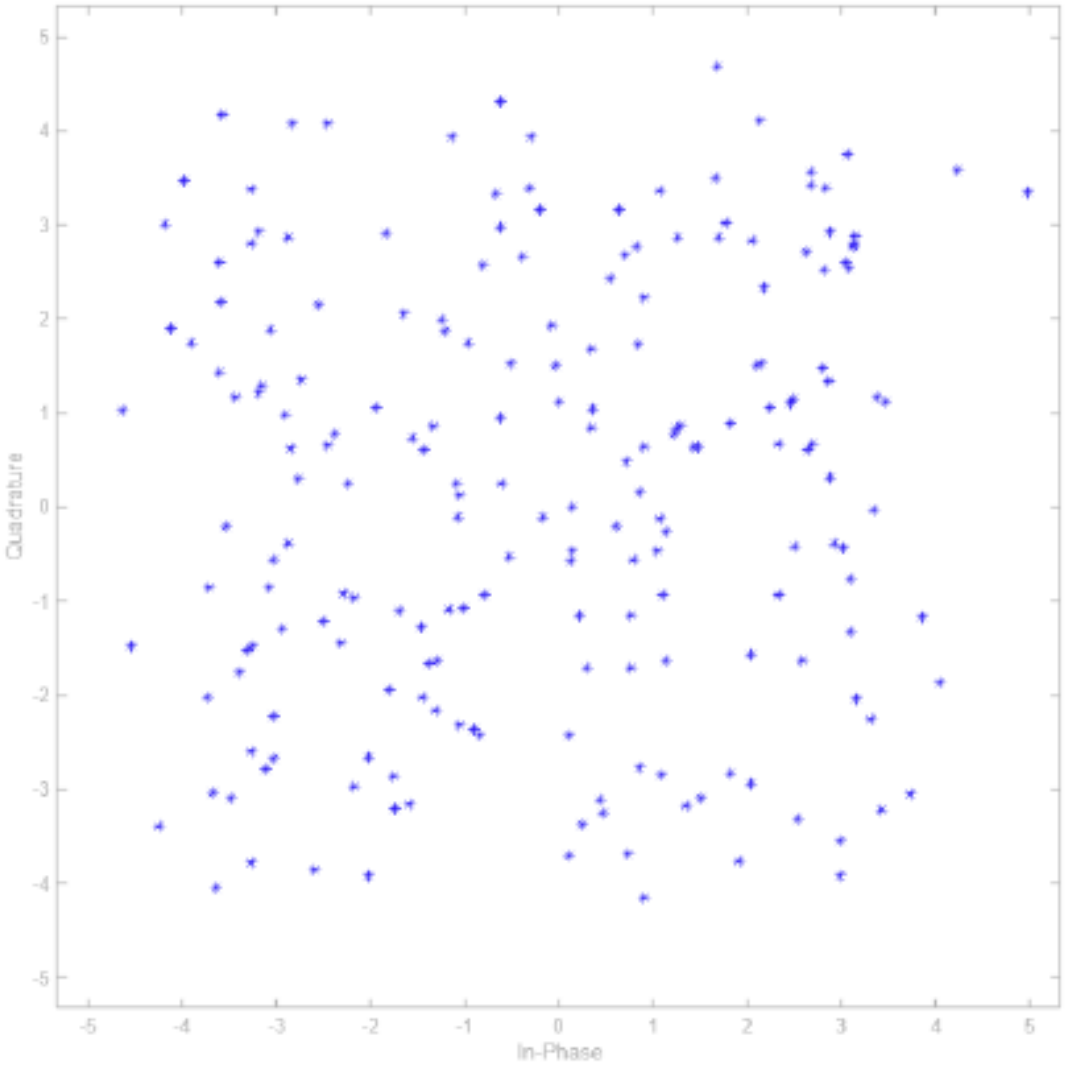




We can observe that the high frequencies have been eliminated.

11. (5) Sample the output of the matched filters at the appropriate time instants and plot the output sequence using the scatterplot command.

```
260     %% A.11
261
262     XIt_demodulated_sampling= XIt_demodulated((2*A_SRRC*T/Ts)+1:over:length(XIt_demodulated)-(2*A_SRRC*T/Ts));
263     XQt_demodulated_sampling= XQt_demodulated((2*A_SRRC*T/Ts)+1:over:length(XQt_demodulated)-(2*A_SRRC*T/Ts));
264
265     for i=1:N
266         sampling(i,1)=XIt_demodulated_sampling(i);
267         sampling(i,2)=XQt_demodulated_sampling(i);
268     end
269     scatterplot(sampling)
270
271     % Get the current figure handle
272     hFig = gcf;
273     % Set the figure background color to white
274     set(hFig, 'Color', [1 1 1]);
275
276     % Get the current axes handle
277     hAxes = gca;
278     % Set the axes background color to white
279     set(hAxes, 'Color', [1 1 1]);
280
```



By executing the above code segments for N=200 and SNR of 10dB, the following experimental error results are obtained:

- Number of symbol errors (errors_num): 54
- Number of bit errors (errors_in_bits): 388

12. (10) Write a function function est X = detect 4 PAM(Y,A) which uses the nearest-neighbor rule and makes decisions for the input sequence of 4-PAM symbols on a symbol-by-symbol basis. Apply the function separately to the samples obtained at the inphase output and at the quadrature output, and obtain decisions for the corresponding input sequences.

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB3/detect_4_PAM.m

```
function [est_x] = detect_4_PAM(Y,A)
    Y_size = size(Y,2);
    est_x = zeros(1,Y_size);
    x = zeros(1,4);

    symbols = [-3*A, -A, A, 3*A];
    for i = 1:Y_size
        distances = zeros(1,4);
        for j = 1:4
            distances(j) = power((Y(i)-symbols(j)),2);
        end
        [~, minIndex] = min(distances);
        est_x(i) = symbols(minIndex);
    end
end
```

13. (10) Using the input sequences and the decisions, compute the number of symbol decision errors for the 16-QAM constellation.

```
%% A.13

x_sent = [XIn; XQn];
x_detected = [detected_XI; detected_XQ];

errors_num = 0;
for i = 1:N
    for j = 1:2
        difference(j,i) = x_sent(j,i) - x_detected(j,i);
        if (difference(j,i) ~= 0)
            errors_num = errors_num + 1;
        end
    end
end
errors_num

```

14. (10) Write a function function est bit = PAM 4 to bits(X,A) which uses the inverse Gray mapping, that is, the mapping from symbols to pairs of bits, and, from the decisions for the input symbol sequences, computes the estimated input binary sequence.

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB3/PAM_4_to_bits.m

```
function [est_bit] = PAM_4_to_bits(X,A)

for i = 1:length(X)
        if X(i) == 3*A
            est_bit(2*i-1:2*i) = [0; 0];
        elseif X(i) == A
            est_bit(2*i-1:2*i) = [0; 1];
        elseif X(i) == -A
            est_bit(2*i-1:2*i) = [1; 1];
        elseif X(i) == -3*A
            est_bit(2*i-1:2*i) = [1; 0];
        end
end
end
```

15. (10) Compute the number of bit errors.

```
%% A.15

bitXI_received = PAM_4_to_bits(detected_XI, A);
bitXQ_received = PAM_4_to_bits(detected_XQ, A);

estimated_bits = [bitXI_received bitXQ_received];
difference = estimated_bits - bits;
errors_in_bits = 0;
for i = 1:4*N
        if (difference(i) ~= 0)
            errors_in_bits = errors_in_bits + 1;
        end
end
errors_in_bits
```

1. For SNR_{dB} = [0 : 2 : 16], experimentally compute the symbol-decision and bit error probabilities by repeating the steps above K times (for example, K = 200, 1000) for each SNR. The experimental estimate of the symbol error probability for each SNR is computed as follows:

```
clear all;
close all;

%% Variable decleration
N = 200;
A = 1;
A_SRRC = 4;
a = 0.5;
T = 0.01;
over = 10;
Ts = T/over;
Fs = 1/Ts;
Nf = 2048;
K = 200;

SNRdb = 0:2:16;
%% B.1
for i = 1:length(SNRdb)
    SNR = SNRdb(i);
    sum_errors_num = 0;
    sum_errors_in_bits = 0;
    for k = 1:K
        bits = (sign(randn(4*N, 1)) + 1)/2;

        % For bits in the [0,2N]
        bitXI = bits(1:2*N);
        XIn = bits_to_4_PAM(bitXI, A);

        % For bits in the [2N+1,4N]
        bitXQ = bits(2*N+1:4*N);
        XQn = bits_to_4_PAM(bitXQ, A);

        % Creating the pulse
        [phi, t] = srrc_pulse(T, over, A_SRRC, a);

        % Upsample the 2 sequencies
        XIn_up = (1/Ts)*upsample(XIn, over);
        XI_up_time = 0:Ts:N*T-Ts;

        XQn_up = (1/Ts)*upsample(XQn, over);
        XQ_up_time = XI_up_time;

        % Convolutions and time
        XIt = Ts*conv(XIn_up, phi);
        XQt = Ts*conv(XQn_up, phi);
        conv_t = t(1) + XQ_up_time(1):Ts:t(end) + XQ_up_time(end);

        % Frequency Range
        f_axis = linspace(-Fs/2,(Fs/2-Fs/Nf), Nf);

```

```
% Periodograms creation
XIF = fftshift(fft(XIt, Nf)*Ts);
num = power(abs(XIF),2);
T_total = conv_t(end) - conv_t(1);
PXIF = num/T_total;

XQF = fftshift(fft(XQt, Nf)*Ts);
num = power(abs(XQF),2);
T_total = conv_t(end) - conv_t(1);
PXQF = num/T_total;

Fo = 200;
XImod_t = 2*XIt.*cos(2*pi*Fo*conv_t);
XQmod_t = -2*XQt.*sin(2*pi*Fo*conv_t);

% Periodograms creation
XIFmod = fftshift(fft(XImod_t, Nf)*Ts);
num = power(abs(XIFmod),2);
T_total = conv_t(end) - conv_t(1);
PXIFmod = num/T_total;

XQFmod = fftshift(fft(XQmod_t, Nf)*Ts);
num = power(abs(XQFmod),2);
T_total = conv_t(end) - conv_t(1);
PXQFmod = num/T_total;

% Input signal creation
Xmod_t = XImod_t + XQmod_t;

% Periodogram creation
Xmod_F = fftshift(fft(Xmod_t, Nf)*Ts);
num = power(abs(Xmod_F),2);
T_total = conv_t(end) - conv_t(1);
PXmod = num/T_total;

% Noise creation
variance = (10*(A^2))/(Ts*(10^(SNR/10)));
gaussian_noise = sqrt(variance)*randn(1,length(Xmod_t));

% Adding noise to the waveform
Xmod_noise = Xmod_t + gaussian_noise;

% Demodulating the signal
XImod_noise = Xmod_noise.*cos(2*pi*Fo*conv_t);
XQmod_noise = -Xmod_noise.*sin(2*pi*Fo*conv_t);

% Periodograms creation
XImod_noise_F = fftshift(fft(XImod_noise, Nf)*Ts);
num = power(abs(XImod_noise_F),2);
T_total = conv_t(end) - conv_t(1);
PXImod_noise = num/T_total;
```

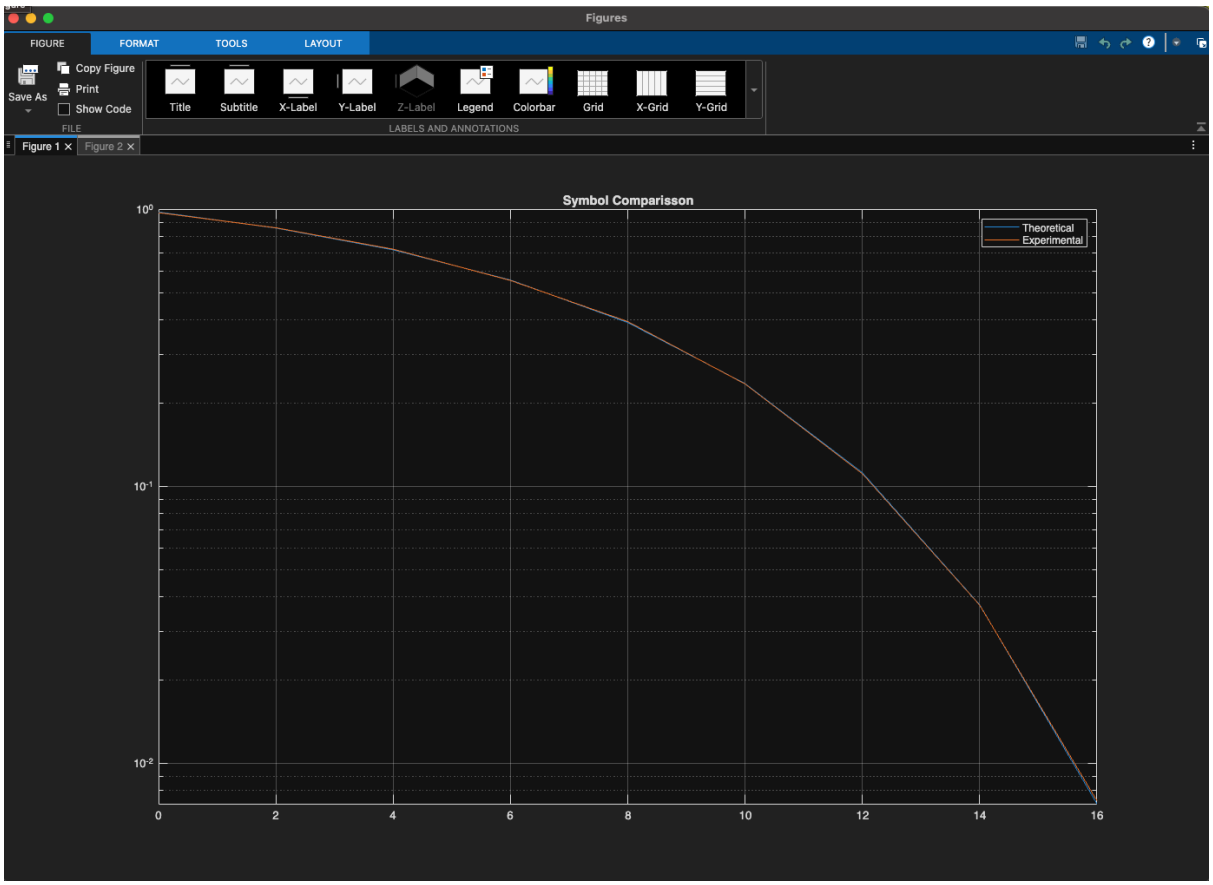
```
103          XQmod_noise_F = fftshift(fft(XQmod_noise, Nf)*Ts);
104          num = power(abs(XQmod_noise_F),2);
105          T_total = conv_t(end) - conv_t(1);
106          PXQ_noise = num/T_total;
107
108          % Filter the waveforms
109          XIt_demodulated = Ts*conv(XImod_noise, phi);
110          XQt_demodulated = Ts*conv(XQmod_noise, phi);
111          conv_t2 = t(1) + conv_t(1):Ts:t(end) + conv_t(end);
112
113          XIF_demodulated = fftshift(fft(XIt_demodulated, Nf)*Ts);
114          num = power(abs(XIF_demodulated),2);
115          T_total = conv_t2(end) - conv_t2(1);
116          PXI_demodulated = num/T_total;
117
118          XQF_demodulated = fftshift(fft(XQt_demodulated, Nf)*Ts);
119          num = power(abs(XQF_demodulated),2);
120          T_total = conv_t2(end) - conv_t2(1);
121          PXQ_demodulated = num/T_total;
122
123          XIt_demodulated_sampling= XIt_demodulated((2*A_SRRC*T/Ts)+1:over:length(XIt_demodulated)-(2*A_SRRC*T/Ts));
124          XQt_demodulated_sampling= XQt_demodulated((2*A_SRRC*T/Ts)+1:over:length(XQt_demodulated)-(2*A_SRRC*T/Ts));
125
126          for p=1:N
127              sampling(p,1)=XIt_demodulated_sampling(p);
128              sampling(p,2)=XQt_demodulated_sampling(p);
129          end
130
131          % Estimating the symbols value
132          for j = 1:N
133              detected_XI(j) = detect_4_PAM(XIt_demodulated_sampling(j), A);
134              detected_XQ(j) = detect_4_PAM(XQt_demodulated_sampling(j), A);
135          end
136
137          x_sent = [XIn; XQn];
138          x_detected = [detected_XI; detected_XQ];
139
140          errors_num = 0;
141          for n = 1:N
142              for j = 1:2
143                  if (x_sent(j,n) ~= x_detected(j,n))
144                      errors_num = errors_num + 1;
145                  end
146              end
147          end
148
149          bitXI_received = PAM_4_to_bits(detected_XI, A);
150          bitXQ_received = PAM_4_to_bits(detected_XQ, A);
151
152          estimated_bits = [bitXI_received bitXQ_received];
153
154          errors_in_bits = 0;
```

Run to Here
Run up to this line and pause

```
            for m = 1:length(estimated_bits)
                if (estimated_bits(m) ~= bits(m))
                        errors_in_bits = errors_in_bits + 1;
                end
            end
        sum_errors_num = sum_errors_num + errors_num;
        sum_errors_in_bits = sum_errors_in_bits + errors_in_bits;
        end
        P_symbol(i) = sum_errors_num/(N*K);
        P_bit(i) = sum_errors_in_bits/(4*N*K);
        P_theor_symbol(i) = 3*Q(sqrt(0.2.*(10.^(SNR/10))));
        P_theor_bit(i) = P_theor_symbol(i)/4;
    end
    %% B.2 and B.3
    figure;
    semilogy(SNRdb, P_theor_symbol);
    hold on;
    semilogy(SNRdb, P_symbol);
    legend ("Theoretical","Experimental")
    hold off;
    grid on;
    title("Symbol Comparisson");

    figure;
    semilogy(SNRdb, P_theor_bit);
    hold on;
    semilogy(SNRdb, P_bit);
    legend ("Theoretical","Experimental")
    hold off;
    grid on;
    title("Bits Comparisson");
```

2. (20) Plot, using semilogy, the theoretical and experimental symbol error probabilities as a function of SNR_{dB}. What do you observe?



We observe that the experimental and theoretical values are almost identical.

3. (20) Plot, using semilogy, the experimental bit error probability as a function of SNR_{dB}. In the same figure, also plot the theoretical approximation for the bit error probability. What do you observe? Can you explain the phenomenon?

